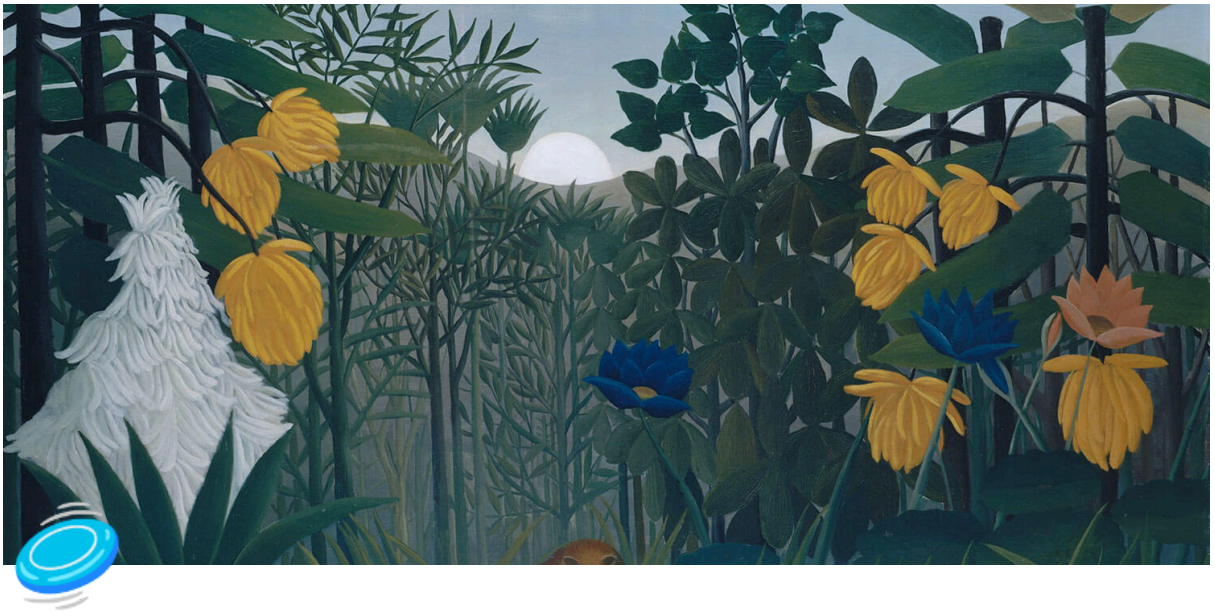


2023 Paper

Question 1

Question 2

- part i
  - Conducting Research to understand the domain
  - Engaging with Relevant Stakeholders
  - Impact on XML component of an information system:
- part ii
  - London Ambulance Service Information System Disaster:
  - THERAC-25 Radiation Therapy Machine Accidents:
  - General implications of not consulting with stakeholders
- part iii

Question 3

Question 1

The C function below implements matrix multiplication. *A*, *B* and *R* are two-dimensional arrays of signed, 16-bit values (short). *A* has dimensions *N*×*M*, *B* has dimensions *M*×*P* and *R* has dimensions *N*×*P*. All arrays are stored in row-major order.

```
void matmul(unsigned int N, unsigned int M, unsigned int P,
            short A[][M], short B[][P], short R[][P])
{
    unsigned int i, j, k;
    short r;
    for (i = 0; i < N; i++)
        {
        for (j = 0; j < P; j++)
                {
                r = 0;
                  for (k = 0; k < M; k++) {
                r += A[i][k] * B[k][j];
                }
                    R[i][j] = r;
                }
        }
}
```

1. Translate the C function shown above into ARM Assembly Language. Do not change the algorithm. Adhere as closely as you can to the ARM Architecture Procedure Call Standard (AAPCS). Include pseudocode comments alongside your implementation. Highlight and explain how you have adhered to the AAPCS.
2. Calculate, as a function of *N, M and P*, the number of memory loads and stores that result
   from reading or modifying the elements of *A*, *B* and *R* in the above function.
3. If we assume that the above function is to be executed on a 32-bit processor, with a 32-bit
   wide processor-memory data bus, then we can load or store two short values using a single
   LDR or STR instruction.

   Discuss the applicability of the above optimisation to each of the arrays *A*, *B* and *R*. Use
   pseudocode to illustrate how you would implement the above modification. What are the
   implications, if any, of the use of the (signed) short data type. What are the potential

performance benefits and costs of your proposed implementation? (You do not need to
provide an assembly language listing for your modified implementation.)

Attempt part 1 and 3 are on paper.

Attempt part 2:

- Memory Load
  - Load elements from array ‘A’ requires 1 load per iteration of the innermost loop
  - Load elements form array ‘B’ requires 1 load per iteration of the middle loop
  - No additional Load for array ‘R’
- Memory Store
  - Storing element into array ‘R’, requires 1 store per iteration of the innermost loop.

Question 2

part i

Two key recommended practices when designing an information system for a particular domain are 1) Conduct research to understand the domain and 2) Engage with all relevant stakeholders.
Discuss how standards-based approaches like Unified Modelling (UML) can support these two recommended practices when designing an information system. In your answer make reference to

- The role of use case diagrams
- Class diagrams and the impact they have on an XML component of an information system.

Attempt:

Conducting Research to understand the domain

use case diagram:

- role: use case diagrams in Unified Modelling Language (UML) help in capturing and visualising the functional requirements of a system from the perspective of

its users.

- support for research: by creating use case diagrams, system designers can identify and document the various interactions between users and the system. This includes understanding the main functionalities, user roles, and how users interact with the system to achieve their gaols. Use case diagrams provides a high-level view of the system's behaviour, making it easier to comprehend and communicate the key aspects of the domain.

Engaging with Relevant Stakeholders

use case diagram:

- Support for Stakeholder Engagement: use case diagrams provides a visual representation of the system's functionality from a user's perspective. This visual representation facilitates discussions with stakeholders by providing a common understanding of how the system will be used to meet their needs. Stakeholders can actively participate in the identification and validation of use cases, ensuring that the system's requirements align with their expectations.

class diagrams:

- Role: class diagrams in UML represent the static structure of a system, showing the classes, their attributes, relationships, and methods.
- Support for Research: During the research phase, class diagrams can be used to model the key concept and entities in the domain. By identifying classes and their relationships, designers gain insights into the structure of the information system and underlying data model. Class diagrams serve as a foundation for understanding the key components and entities that need to be considering in the system.

Impact on XML component of an information system:

- XML (eXtensible Markup Language): XML is often used for representing structured data in information systems.
- Class Diagram and XML: Class diagrams can influence the XML representation by guiding the definition of XML schema or a DTD. Classes in the class diagram can be mapped to XML elements, attributes to represent properties, and associations to represent relationships between elements. This mapping ensures that the XML structure reflects the conceptual model defined in the class diagram.

- Example impact: if a class diagram identifies a “Person” class with attributes like “Name” and “Address”, corresponding XML schema might define <Person> element with child element <Name> and <Address>. Associations in class diagrams can also influence the structure and nesting of XML elements.

Considerations for the use of UML:

- Clarity and communication: UML diagrams, including use case and class diagrams, provide a standardised and visual way to communicate complex concepts to both technical and non-technical stakeholders. This is essential for effective engagement with stakeholders and ensuring a shared understanding of the domain.
- Flexibility: UML is flexible and can be adapted to different domains and methodologies. It allows designers to model various aspects of a system, from high-level requirements to detailed structural and behavioural specifications.

In summary, UML, particularly through the use of use case and class diagrams, plays an important role in supporting both the research to understand the domain and engagement with relevant stakeholders during the design of an information system.

part ii

Drawing from the London Ambulance Service information system disaster, the THERAC disaster and/or other information systems with which you are familiar discuss the **implications of not consulting with all relevant stakeholders during all stages of information system design, development and deployment**.

Attempt:

London Ambulance Service Information System Disaster:

Background:

The London Ambulance Service faced a significant information system disaster in 1992. The system, designed to improve ambulance dispatching, suffered from a range of issues, including software bugs and inadequate testing.

Implications:

1. Operational impact:
    a. Over-reliance on technology: The LAS system was implemented without thorough consultation with paramedics and other stakeholders. This

resulted in an over-reliance on the automated dispatch system, which couldn't handle the complexity of real world emergencies.

   b. Service Disruption: The system failed during a major incident, leading to service disruption and delays in responding to emergency calls. Lives were potentially at risk due to the breakdown of the information system.

2. Stakeholder Trust and Resistance:

   a. Lack of User Involvement: Paramedics were not adequately involved in the design and testing phases. The lack of user involvement led to a system that did not align with the operational needs and workflows of the ambulance service.

   b. Resistance to Adaption: Due to the system's inadequacy, there was resistance from paramedics to adopt the technology. This resistance stemmed from a lack of understanding of user needs and the practical challenge faced by paramedics in the field.

3. Financial Consequence:

   a. Resource Wastage: The failure of the LAS system resulted in significant financial losses. Resources invested in the development in the implementation of the system were wasted due to the need for emergency manual dispatching and subsequent system abandonment.

THERAC-25 Radiation Therapy Machine Accidents:

Background:

The THERAC-25 incident occurred in 1980 and involved a medical linear accelerator used for the radiation therapy. Software-related malfunctions led to several accidents, resulting in patient injuries and death.

Implications:

1. Patient Safety:

   a. Inadequate Safety Mechanisms: The THERAC-25 incidents were caused by a lack of safety interlocks in the software, which could have prevent hazardous conditions. Failure to consult with medical professionals and safety experts led to a system that did not adequately prioritise patient safety.

2. Lack of Redundancy and Safeguards:

   a. Design flaws: The absence of adequate consultation with radiation therapy experts and the exclusion of safety mechanisms in the software highlighted

a lack of redundancies and safeguards in the system design

   b. Multiple Accidents: The software design flaws resulted in multiple accidents where patient received excessive radiation doses, illustrating the critical importance of involving all relevant stakeholder in the safety-critical domain of medical devices.

3. Legal and Ethical Ramifications:

   a. Liability and Lawsuits: The accidents led to legal action and lawsuit against the manufacturer. The lack of stakeholder consultation contributed to a failure in foreseeing potential risks, exposing the organisation to legal and ethical consequence.

General implications of not consulting with stakeholders

1. Mismatch with User needs:

   a. Failure to consult with end-user and domain experts can lead to systems that do not align with actual user needs and operational requirements.

2. Risk of Unintended Consequences:

   a. Lack of input from diverse stakeholders increases the risk of overlooking potential consequences, including safety issues, security vulnerabilities, and operational disruptions.

3. Resistance to new changes:

   a. Stakeholders who are not consulting may resist adopting or using a new system, especially if it disrupt established workflows or does not address their concerns.

4. Financial losses:

   a. Information systems that do not meet the expectations of stakeholders may lead to financial losses due to system failures, rework, or abandonment.

part iii

Using semantic web based technologies brings opportunities and challenges when linking data from many diverse datasets owned by different organisations.
Using examples, discuss opportunities and challenges that exist for developers at each layer of the semantic web stack.

1. RDF Schema and OWL layer:

a. Opportunities:

i. Data inference: RDF Schema (RDFs) and Web Ontology Language enable developers to define relationships and constraints , allowing for automated reasoning and inference, This can enhance data understanding and discover implicit relationships.

ii. Standardised Ontologies: Reusing established ontologies fosters a shared understanding of concepts across datasets, promoting consistency and alignment.

b. Challenges:

i. Ontology Mapping: Different organisations may use different Ontologies or have varying interpretations of shared ontologies. Harmonising these differences requires efforts and consensus.

ii. Learning Curve: understanding and working with complex ontologies, especially OWL, can be challenging for developers. Training and expertises are crucial.

2. Web standards and Protocols Layer:

a. Opportunities:

i. Interoperability: Standard Web protocols such as HTTP, HTTPS and content negotiation promote interoperability and facilitate communication between diverse system

ii. Decentralised Architecture: The decentralise nature of the web, coupled with standards like HTTP, supports the distributed publishing and consumption of linked data.

b. Challenges:

i. Security and Privacy concerns: Exposing data through web protocols raises security and privacy concerns. Implementing secure communication, access controls, and encryption is essential.

ii. Adoption and Compliance: Ensuring widespread adoption and compliance with web standards can be challenging, as organisations may have different infrastructures and technologies.

3. Resource Description Framework (RDF) Layer:

a. Opportunities:

i. Data interoperability: RDF allows developers to model data in a standardised format, enabling interoperability across diverse datasets. Different organisations can represent their data using RDF, ensuring a common understanding.

ii. Flexibility Data representation: RDF provides flexibility in representing relationships and attributes, accommodating a wide range of data structure and types.

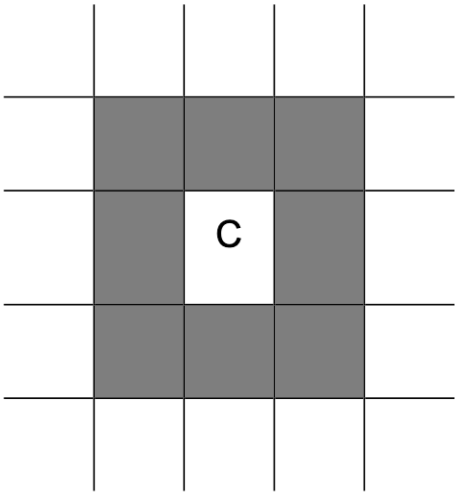
b. Challenges:

i. Data Quality and Consistency: Ensuring data quality and consistency across different datasets can be challenging, different in data models, naming conventions, or schema may arise requiring careful mapping and alignment,

ii. Scalability: As the volume of linked data grows, managing and querying large RDF datasets can become computationally expensive, optimising queries and data storage becomes crucial.

Question 3

The Little Game of Life consists of a 64 × 64 grid of square cells. Each cell is in one of two states: live or dead. Every cell interacts with its eight neighbours, that is the cells that are horizontally, vertically, or diagonally adjacent. The picture shows a cell C, and its eight neighbouring cells, which are highlighted in grey.



On each time step, the value of each cell is updated with the following five rules:

1. Any live cell with fewer than two live neighbours dies.
2. Any live cell with two or three live neighbours remains alive.
3. Any live cell with more than three live neighbours dies.
4. Any dead cell with exactly three live neighbours becomes a live cell.
5. All other dead cells remain dead.

For those cells on the edge (or corner) of the 64 × 64 grid, neighbouring cells that are outside the 64 × 64 grid should always be treated as dead.

The 64 × 64 grid for the can be represented with the following structure:

```
struct grid64x64 {
    uint64_t cells[64];
};
```

Each of the 64 elements of the cells array consists of a 64-bit unsigned integer that can be used to represent one row of the 64 × 64 grid. Each bit of the 64-bit integer can be used to represent the state of one cell of the grid, with 1 representing live, and 0 representing dead.

Write a C function with the following prototype:

```
struct grid64x64 * grid64x64_next_state(struct grid64x64 * grid);
```

In the prototype, grid is a pointer to the current state of the grid. Your function should return a newly allocated grid64x64 value that contains the new state of the grid after the five update rules have been applied to each cell of the current state.

Your code should be written using good programming style, with appropriate use of comments and indentation. Please comment on any assumptions that you make about the order of bits in your solution.

Attempt:

Code is written on paper.

Assumptions:

- The order of bits within each uint64_t in the cells array is assumed to be from the least significant bit (rightmost) to the most significant bit (leftmost)
- The function uses malloc, it is assume the caller will free the memory after to avoid memory leaks.